



GROUP BY and ORDER BY

This SQL script demonstrates **data aggregation and sorting techniques** using the employees and job_roles tables in the techsphere_solutions database.

It shows how SQL is used for **summarizing data (GROUP BY)** and **organizing results (ORDER BY)** for analysis and reporting.

1. Database Selection

```
USE techsphere_solutions;
```

What it does:

- Selects the active database
- Ensures all queries run on the correct schema

2. GROUP BY Clause (Data Aggregation)

2.1 Basic grouping of employees by department

```
SELECT department_id  
FROM employees  
GROUP BY department_id;
```

What it does:

- Groups employees by department
- Returns unique department IDs only

2.2 Count employees per department

```
SELECT department_id, COUNT(*) AS total_employees  
FROM employees  
GROUP BY department_id;
```

What it does:

- Groups employees by department
- Counts how many employees belong to each department



2.3 Average age by gender

```
SELECT gender,  
AVG(TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE())) AS  
average_age  
FROM employees  
GROUP BY gender;
```

What it does:

- Groups employees by gender
- Calculates average age using birth date

2.4 Unique job roles

```
SELECT role_title  
FROM job_roles  
GROUP BY role_title;
```

What it does:

- Returns distinct job roles
- Removes duplicates

2.5 Role and salary combinations

```
SELECT role_title, base_salary  
FROM job_roles  
GROUP BY role_title, base_salary;
```

What it does:

- Groups unique combinations of role and salary



2.6 Salary statistics per role

```
SELECT role_title,  
MIN(base_salary),  
MAX(base_salary),  
AVG(base_salary),  
COUNT(*)  
FROM job_roles  
GROUP BY role_title;
```

What it does:

- Provides salary insights per role:
 - Minimum salary
 - Maximum salary
 - Average salary
 - Number of roles

3. ORDER BY Clause (Sorting Data)

3.1 Sort employees A–Z

```
ORDER BY first_name;
```

What it does:

- Sorts employee names alphabetically (ascending)

3.2 Sort employees Z–A

```
ORDER BY first_name DESC;
```

What it does:

- Sorts names in reverse alphabetical order

3.3 Sort by department then birth date

```
ORDER BY department_id, date_of_birth;
```

**What it does:**

- First sorts by department
- Then sorts employees within each department by age

3.4 multi-column descending sort

```
ORDER BY department_id DESC, date_of_birth DESC;
```

What it does:

- Sorts departments in reverse order
- Within each department, sorts by newest birth dates first

3.5 Sort job roles by salary

```
ORDER BY base_salary DESC;
```

What it does:

- Displays highest-paying roles first

4. ORDER BY Using Column Position

```
ORDER BY 3, 4;
```

What it does:

- Sorts using column positions instead of names
- Column 3 = department_id
- Column 4 = date_of_birth

⚠ Not recommended in production because structure changes can break queries.

5. Combining GROUP BY + ORDER BY**5.1 Employee count per department (sorted)**

```
ORDER BY total_employees DESC;
```

**What it does:**

- Groups employees by department
- Counts them
- Sorts departments from largest to smallest

5.2 Average salary per role (sorted)

```
ORDER BY avg_salary DESC;
```

What it does:

- Calculates average salary per role
- Sorts roles from highest to lowest pay

6. Overall Purpose of the Script

This script demonstrates how SQL is used to:

- Group data into meaningful categories
- Perform aggregate calculations
- Analyze workforce structure
- Sort results for reporting and dashboards
- Combine grouping and sorting for business insights

7. Key Learning Outcomes

After running this script, you understand:

- How GROUP BY aggregates data
- How ORDER BY sorts results
- How to combine both for analytics
- How to generate business insights from raw data

8. Summary Insight

GROUP BY is used for **summarizing data**, while ORDER BY is used for **structuring results for readability and analysis**. Together, they form the foundation of SQL reporting and business intelligence queries.